3D Object Recognition

3D Object Recognition
- 1. Content Description
- 2. Program Startup
- 3. Core Code Analysis

1. Content Description

This lesson implements acquiring the color image and using the MediaPipe framework to recognize 3D objects (the cases demonstrate four objects only: mug, shoe, chair, and camera). This section requires entering commands in the terminal, where the terminal is opened according to the mainboard type. This lesson uses the Raspberry Pi 5 as an example.

For the Raspberry Pi 5 mainboard, you need to open a terminal on the host machine, then enter the command to access the Docker container, and enter the commands mentioned in this lesson inside the terminal after entering the Docker container. For the tutorial on entering the Docker container from the host, please refer to **【Robot Configuration and Operation Guide】, section 【3.Enter Docker (For Raspberry Pi)】** in this product tutorial.

For the Orin and RDK mainboards, simply open a terminal and enter the commands mentioned in this lesson.

2. Program Startup

The Raspberry Pi PI5 controller needs to enter the Docker container first; RDK X5 and Orin controllers do not need to.

To enter the Docker container (for steps, refer to 【Docker Course】 --- 【4. Docker Startup Script】),

All the docker terminals for the commands below need to enter the same Docker container before executing **(for steps, refer to 【Docker Course】 --- 【3.Docker Submission and Multi-Terminal Access】)**

First, enter the following command in the terminal to start the robot camera and low-level driver environment:

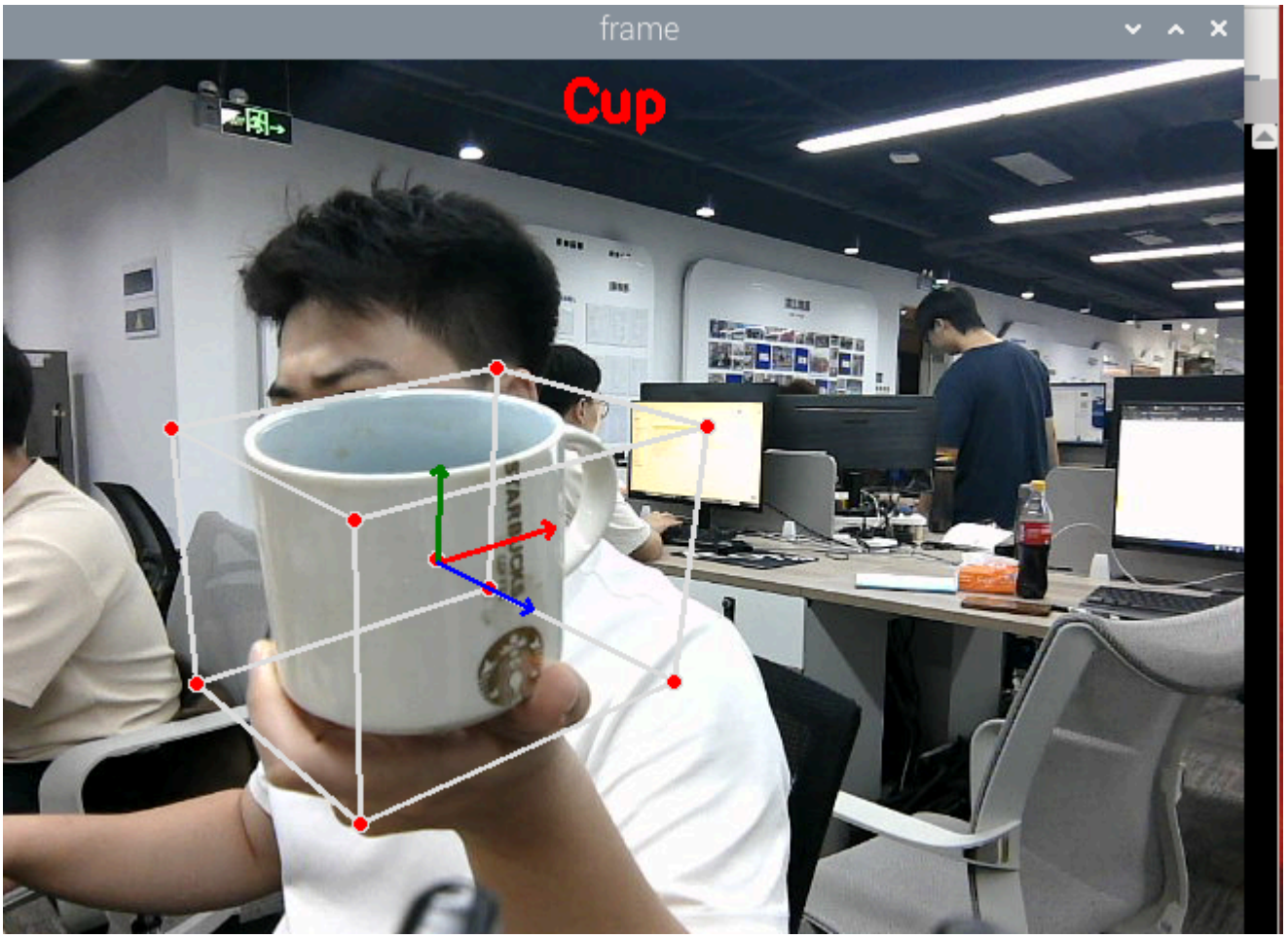
```
ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
```

> **Note**
>
> This command is the official all-in-one startup script for the A1Pro, which automatically launches the image stream node according to the camera hardware configuration (USB monocular camera or Nuwa 3D depth camera) and loads the low-level drivers. If you only need to start the camera stream separately, you can also run `ros2 launch usb_cam camera.launch.py` (USB camera) or `ros2 launch ascamera hp60c.launch.py` (Nuwa camera).

After the camera is started successfully, open another terminal and enter the following command in the terminal to start the 3D object detection program:

```
ros2 run yahboomcar_mediapipe 07_Objectron
# Control the robotic arm to look forward
ros2 topic pub /arm6_joints dofbot_pro_interface/msg/ArmJoints "{joint1: 90, joint2: 180,
joint3: 0, joint4: 0, joint5: 90, joint6: 0, time: 1000}" --once
```

After the program runs, the default recognized object is the Shoe. You can press the F key to switch the recognized object. As shown in the figure below, a mug is recognized:



3. Core Code Analysis

Program code path:

- Raspberry Pi 5 mainboard

  The program code is located in the running Docker. The path in the Docker is `/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/07_Objectron.py`

- Orin mainboard

  The program code path is `/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/07_Objectron.py`

- RDK mainboard

  The program code path is

`/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/07_Objectron.py`

Import the required library files:

```
import mediapipe as mp
import cv2 as cv
import time
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import os
import numpy as np
```

Initialize the data and define the Publisher and Subscriber:

```
def __init__(self, staticMode=False, maxObjects=5, minDetectionCon=0.5,
minTrackingCon=0.99):
    super().__init__('objectron')
    self.staticMode = staticMode
    self.maxObjects = maxObjects
    self.minDetectionCon = minDetectionCon
    self.minTrackingCon = minTrackingCon
    self.index = 0
    self.modelNames = ['Shoe', 'Chair', 'Cup', 'Camera']
    # Use the class in the mediapipe library to define a 3D detection object
    self.mpObjectron = mp.solutions.objectron
    self.mpDraw = mp.solutions.drawing_utils
    self.mpobjectron = self.mpObjectron.Objectron(
    self.staticMode, self.maxObjects, self.minDetectionCon, self.minTrackingCon,
self.modelNames[self.index])
    self.bridge = CvBridge()
    # Define the Subscriber for the color image topic
    camera_type = os.getenv('CAMERA_TYPE', 'usb')
    topic_name = '/ascamera_hp60c/camera_publisher/rgb0/image' if camera_type == 'nuwa' else
'/usb_cam/image_raw'
    self.subscription = self.create_subscription(
    Image,
    topic_name,
    self.image_callback,
    10)
```

Color image callback function:

```
def image_callback(self, msg):
    frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
    action = cv2.waitKey(1)
    # Press the F key to switch the recognized object
    if action == ord('f') or action == ord('F') : self.configUP()
    frame = self.findObjectron(frame)
    cv.imshow('Objectron', frame)
```

The configUP function for switching the recognized object. It selects a value from self.modelNames by modifying the value of self.index; self.modelNames can be selected from ['Shoe', 'Chair', 'Cup', 'Camera']:

```
def configUP(self):
    self.index += 1
    if self.index>=4:self.index=0
        self.mpobjectron = self.mpObjectron.Objectron(
        self.staticMode, self.maxObjects, self.minDetectionCon, self.minTrackingCon,
self.modelNames[self.index])
```

The findObjectron function for object recognition:

```
def findObjectron(self, frame):
    cv.putText(frame, self.modelNames[self.index], (int(frame.shape[1] / 2) - 30, 30),
    cv.FONT_HERSHEY_SIMPLEX, 0.9, (0, 0, 255), 3)
    # Convert the color space of the input image from BGR to RGB for subsequent image
processing
    img_RGB = cv.cvtColor(frame, cv.COLOR_BGR2RGB)
    # Call the process function in the mediapipe library for image processing; during init,
the self.pose object was created and initialized
    results = self.mpobjectron.process(img_RGB)
    # Determine whether the target object has been recognized
    if results.detected_objects:
        for id, detection in enumerate(results.detected_objects):
            # Draw the coordinate system on the recognized target object in the image
            self.mpDraw.draw_landmarks(frame, detection.landmarks_2d,
self.mpObjectron.BOX_CONNECTIONS)
            self.mpDraw.draw_axis(frame, detection.rotation, detection.translation)
    return frame
```